

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

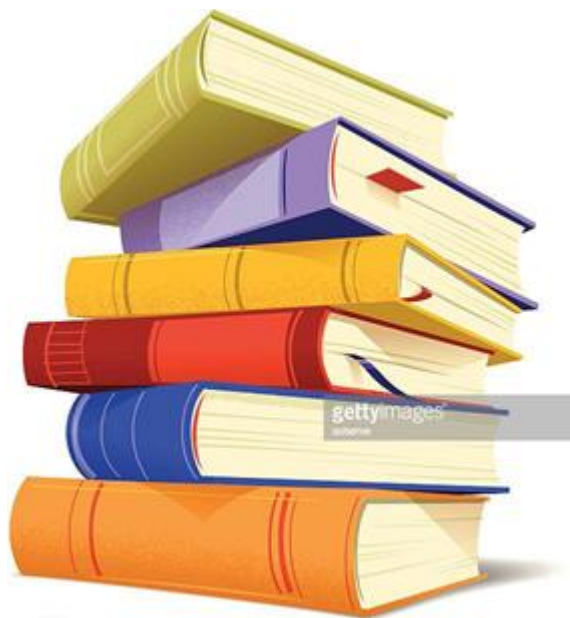
www.hoasen.edu.vn

Chương 3

Giới thiệu Keras và TensorFlow



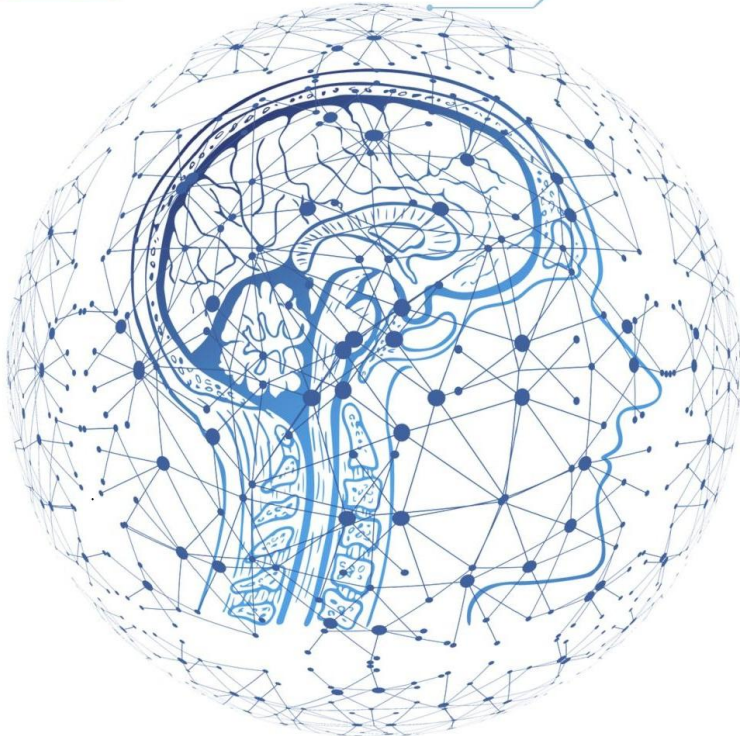
Giới thiệu Keras và TensorFlow



- TensorFlow & Keras
- Keras và TensorFlow – Lịch sử tóm lược
- Thiết lập môi trường cho học sâu
- Quy trình làm việc với Keras
- Tạo các layer và model mới thông qua kế thừa
- Dùng các vòng lặp huấn luyện và đánh giá có sẵn
- Tự xây dựng các vòng lặp huấn luyện và đánh giá

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



TensorFlow

TensorFlow là gì?

■ TensorFlow là

- Một nền tảng tính toán số cho machine learning và deep learning
- Một framework hỗ trợ xây dựng, huấn luyện và triển khai mô hình
- Một hệ sinh thái lớn, không chỉ là một thư viện đơn lẻ

■ TensorFlow hỗ trợ

- Các phép toán trên tensor
- Vi phân tự động

- Tăng tốc bằng GPU / TPU

- Huấn luyện, đánh giá và triển khai

■ Ý tưởng cốt lõi

- Mọi thứ xoay quanh tensor và computation graph
- Framework giúp tối ưu việc tính toán gradient và cập nhật tham số

Vì sao TensorFlow quan trọng trong deep learning?

■ TensorFlow quan trọng vì

- Tự động tính gradient bằng autodiff
- Mở rộng tốt trên phần cứng mạnh
- Hỗ trợ triển khai và đưa mô hình vào sử dụng thực tế
- Tương thích với nhiều API mức cao như Keras

■ Từ góc nhìn người học

- Không cần tự viết backpropagation từ đầu
- Tập trung hơn vào kiến trúc mô hình và dữ liệu
- Có thể đi từ nghiên cứu đến ứng dụng thực tế

TensorFlow & Tensor

■ Nhắc lại từ Chương 2

- Tensor là cấu trúc dữ liệu nhiều chiều
- Deep learning thực chất là chuỗi phép toán tensor

■ Trong TensorFlow

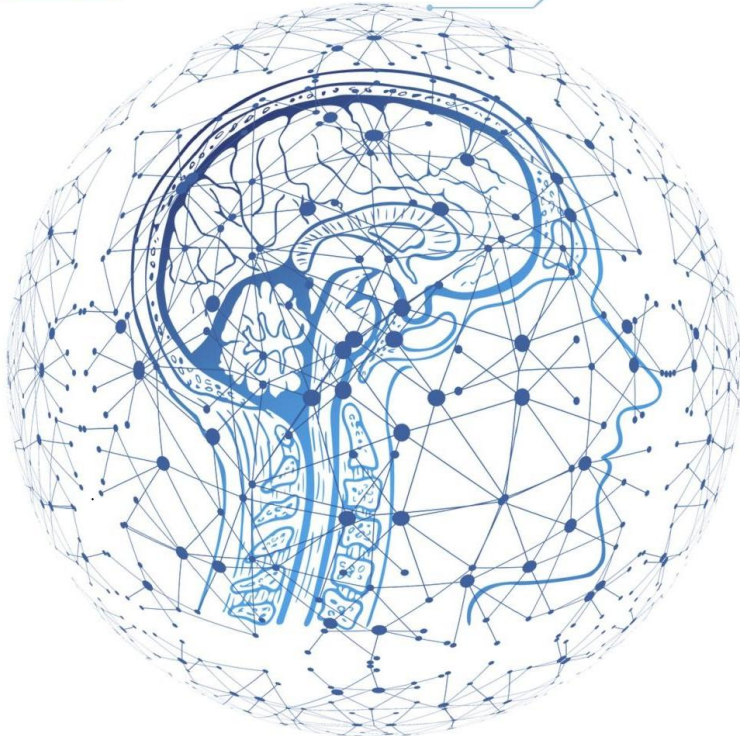
- Tensor là đối tượng trung tâm
- Các phép toán trên tensor có thể được tối ưu trên CPU/GPU/TPU
- Gradient có thể được tính tự động

■ Ví dụ trực quan

- Dữ liệu đầu vào: tensor
- Trọng số mô hình: tensor
- Kết quả dự đoán: tensor
- Gradient: tensor

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Keras

Keras là gì?

■ Keras là

- API mức cao để xây dựng deep learning model
- Giao diện dễ dùng, trực quan, nhất quán
- Tích hợp chặt chẽ với TensorFlow

■ Mục tiêu của Keras

- Giảm độ phức tạp khi xây mô hình
- Giúp thử nghiệm nhanh
- Tăng tính dễ đọc, dễ bảo trì của code

■ Triết lý thiết kế

Progressive disclosure of complexity

(Đơn giản khi mới bắt đầu, nhưng vẫn có thể mở rộng khi cần)

Tại sao dùng Keras thay vì chỉ dùng TensorFlow thuần?

■ Dùng Keras vì

- Code ngắn gọn hơn
- Workflow chuẩn hóa hơn
- Dễ debug và dễ đọc
- Nhiều thành phần dùng lại được

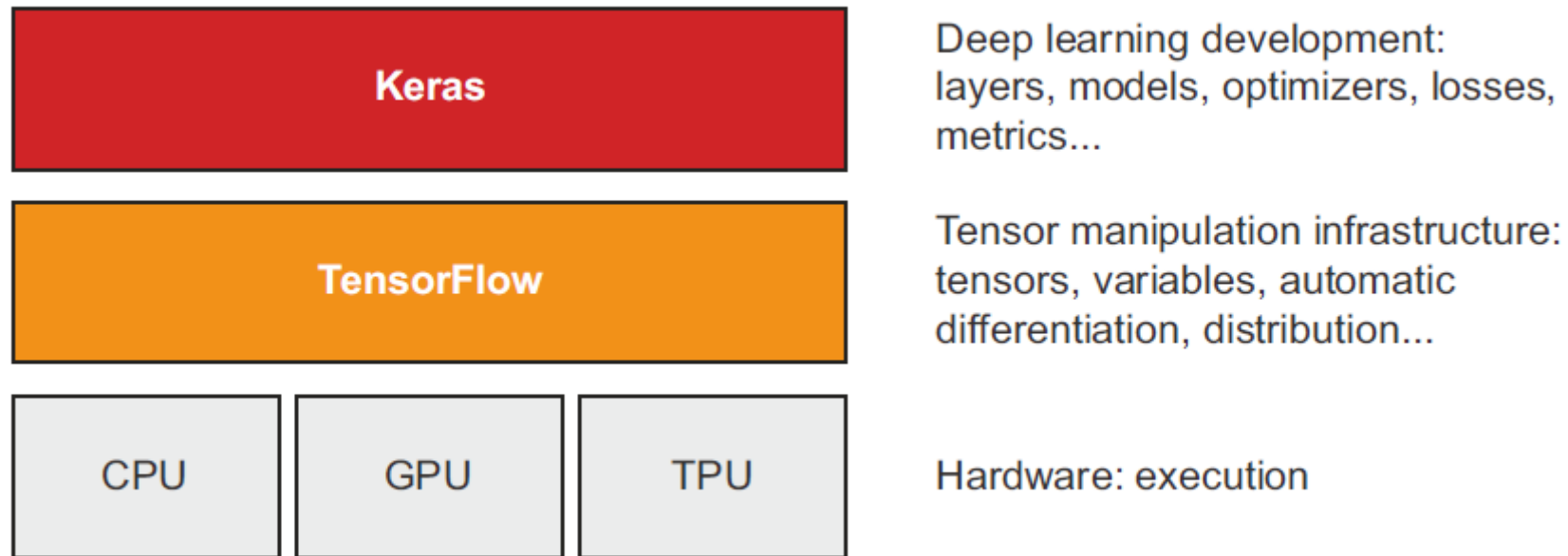
■ So sánh ngắn

- TensorFlow thuần: linh hoạt hơn ở mức thấp
- Keras: thuận tiện hơn ở mức xây model, train, evaluate

■ Thông điệp

- Keras không thay thế TensorFlow
- Keras là giao diện cấp cao chạy trên TensorFlow

Tại sao dùng Keras thay vì chỉ dùng TensorFlow thuần?



- TensorFlow is a low-level tensor computing platform
- Keras is a high-level deep learning API

Các thành phần cơ bản trong Keras

1. Layer

Khối xử lý dữ liệu cơ bản

2. Model

Tập hợp các layer thành một mạng hoàn chỉnh

3. Loss

Đo độ sai giữa dự đoán và nhãn thật

4. Optimizer

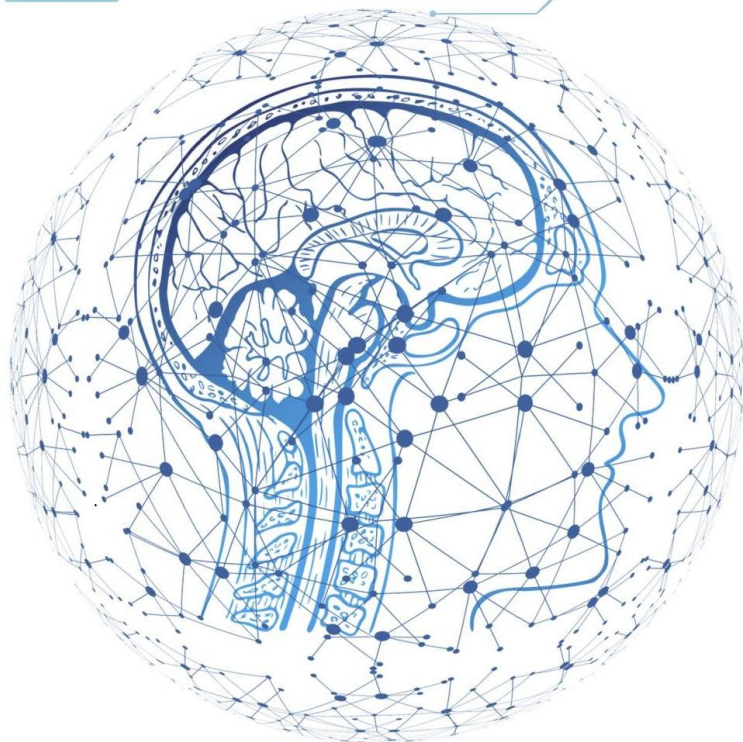
Cập nhật trọng số để giảm loss

5. Metric

Chỉ số để theo dõi hiệu quả

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Keras và TensorFlow Lịch sử tóm lược

Lịch sử hình thành

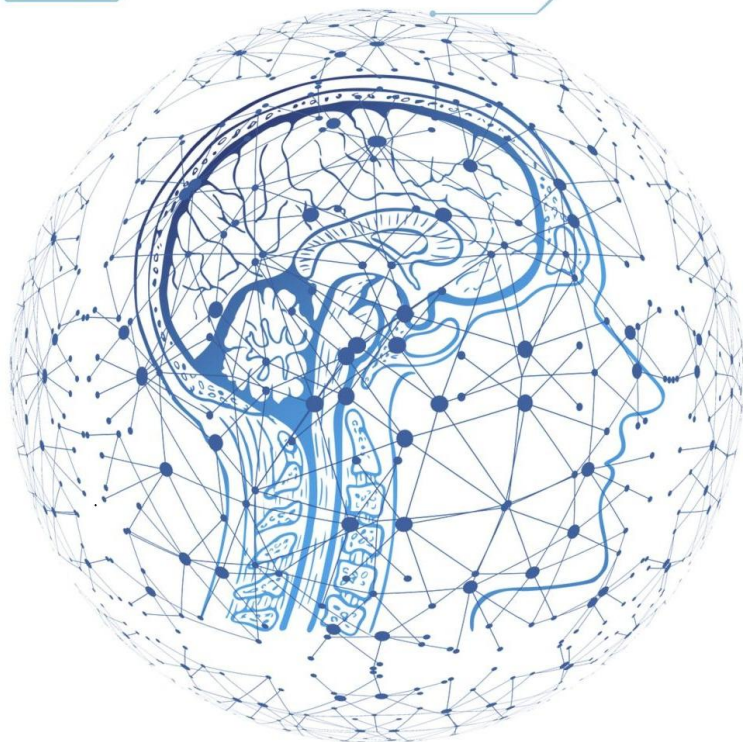
- 03/2015: Keras được phát hành, sớm hơn TensorFlow khoảng 8 tháng.
- 11/2015: TensorFlow được Google giới thiệu.
- Ban đầu, Keras được xây dựng trên Theano, một thư viện hỗ trợ tensor, tự động vi phân và GPU.
- Theano là tiền thân quan trọng, tiên phong trong việc sử dụng **static computation graph**.
- **Giai đoạn cuối 2015**
 - Keras chuyển sang kiến trúc multi-backend.
 - Có thể chạy trên Theano hoặc TensorFlow.
 - Chỉ cần thay đổi biến môi trường để chuyển backend.

Tổng nhất Keras – TensorFlow

- 2016: TensorFlow trở nên ổn định và được chọn làm backend mặc định cho Keras.
- 2017: Keras hỗ trợ thêm CNTK (Microsoft) và MXNet (Amazon).
- **Hiện tại**
 - Theano và CNTK đã ngừng phát triển.
 - MXNet ít phổ biến.
 - Keras quay về chỉ dùng TensorFlow (single-backend).
- **Quan hệ cộng sinh**
 - 2016–2017: Keras giúp TensorFlow dễ tiếp cận hơn với người dùng.
 - 2018: TensorFlow chọn Keras làm high-level API chính thức.
 - 2019: TensorFlow 2.0 ra mắt, tích hợp chặt chẽ Keras.
- **Ý nghĩa**
 - Chuẩn hóa hệ sinh thái Deep Learning.
 - Keras trở thành cách chính để sử dụng TensorFlow.

DEEP LEARNING

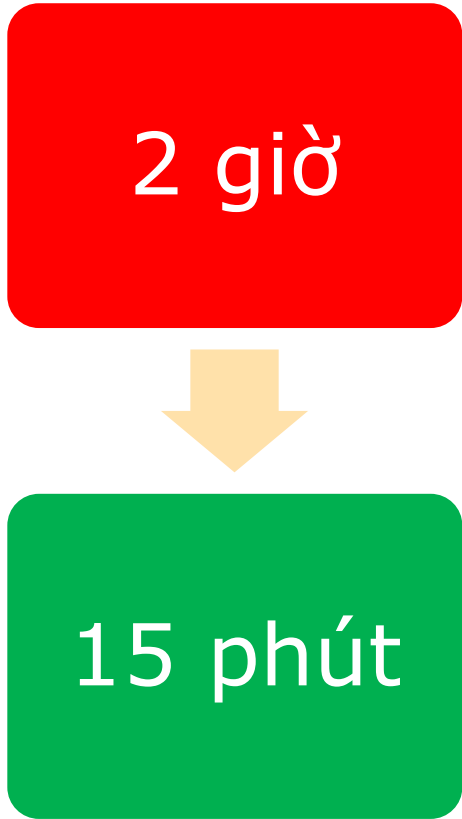
NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Thiết lập môi trường cho học sâu

Yêu cầu môi trường học sâu

- Trước khi phát triển ứng dụng Deep Learning, cần thiết lập môi trường làm việc phù hợp.
- Mặc dù có thể chạy trên CPU, nhưng việc sử dụng GPU được khuyến nghị mạnh mẽ để tăng hiệu năng.
- Các bài toán như xử lý ảnh với CNN sẽ chạy rất chậm trên CPU, kể cả CPU đa nhân.
- Khi sử dụng GPU, tốc độ có thể tăng lên khoảng 5 đến 10 lần so với CPU.



2 giờ

15 phút

Các lựa chọn sử dụng GPU

- Có 3 cách chính để sử dụng GPU cho Deep Learning:
 - **Cách 1:**
 - Cài đặt GPU NVIDIA trên máy cá nhân.
 - Cách này cho hiệu năng ổn định và phù hợp cho người dùng lâu dài.
 - **Cách 2:**
 - Sử dụng cloud như Google Cloud hoặc AWS EC2.
 - Cho phép mở rộng tài nguyên linh hoạt mà không cần mua phần cứng.
 - **Cách 3:**
 - Sử dụng Google Colab với GPU miễn phí.
 - Đây là cách dễ nhất để bắt đầu vì không cần cài đặt hay đầu tư ban đầu.
 - Cấu hình: **Runtime** → **Change runtime type** → **Hardware accelerator** → chọn GPU.

Google Colab – Lựa chọn cho người mới

- Google Colab là dịch vụ notebook chạy trên cloud, hỗ trợ GPU miễn phí.
- Người dùng chỉ cần trình duyệt là có thể bắt đầu lập trình.
- Ưu điểm là dễ sử dụng, không cần cài đặt và phù hợp với việc học hoặc thử nghiệm nhỏ.
- Tuy nhiên, phiên bản miễn phí chỉ phù hợp với workload nhỏ, không phù hợp cho các dự án lớn.

Cloud vs Local – Khi nào nên dùng?

- Cloud phù hợp khi cần mở rộng nhanh hoặc chưa có GPU. Đây là giải pháp chi phí thấp trong ngắn hạn.
- Tuy nhiên, sử dụng cloud lâu dài sẽ tốn kém. Ví dụ, một GPU V100 có thể tốn khoảng 2.48 USD mỗi giờ.
- Trong khi đó, mua GPU cá nhân có giá khoảng 1,500–2,500 USD, phù hợp nếu sử dụng lâu dài.
- Kết luận:
 - Người dùng ít: nên dùng cloud hoặc Colab.
 - Người dùng nhiều: nên đầu tư GPU riêng.

Hệ điều hành khuyến nghị

- Nên sử dụng hệ điều hành Unix (Linux) khi làm Deep Learning.
- Mặc dù có thể chạy trên Windows, nhưng không được khuyến khích vì dễ gặp lỗi và khó cấu hình.
- **Giải pháp cho người dùng Windows:**
 1. Cài dual boot Ubuntu.
 2. Sử dụng Windows Subsystem for Linux 2 (WSL2).
 3. Docker Desktop sử dụng WSL2 làm nền tảng để chạy Linux containers → mô phỏng **distributed system**.
- **Lợi ích:** Giảm lỗi môi trường. Tương thích tốt với thư viện Deep Learning.

Giới thiệu TPU

- TPU (Tensor Processing Unit) là chip chuyên dụng do Google phát triển
 - Được thiết kế tối ưu cho machine learning, đặc biệt là deep learning
 - Tập trung vào xử lý tensor hiệu quả
 - Giúp train model nhanh hơn nhiều so với CPU & GPU
- **So sánh:**
 - CPU: đa năng, xử lý tuần tự
 - GPU: xử lý song song tốt, dùng nhiều trong deep learning
 - TPU: chuyên biệt cho tensor, nhanh hơn GPU trong nhiều bài toán AI
 - **Có thể sử dụng qua:**
 - `tf.distribute.TPUStrategy`
 - Google Colab (TPU miễn phí). Cấu hình: **Runtime** → **Change runtime type**

Vì sao cần TPU

- Deep learning cần tính toán ma trận rất lớn
- GPU đã nhanh nhưng vẫn chưa tối ưu hoàn toàn
- TPU được thiết kế riêng để:
 - tăng tốc nhân ma trận
 - giảm độ trễ
 - tiết kiệm năng lượng
- Khi nào nên dùng TPU:
 - Khi train mô hình lớn
 - Khi dữ liệu lớn
 - Khi cần rút ngắn thời gian huấn luyện
 - Không phù hợp cho mô hình nhỏ hoặc thử nghiệm đơn giản

Ưu/Nhược điểm của TPU

■ Ưu điểm:

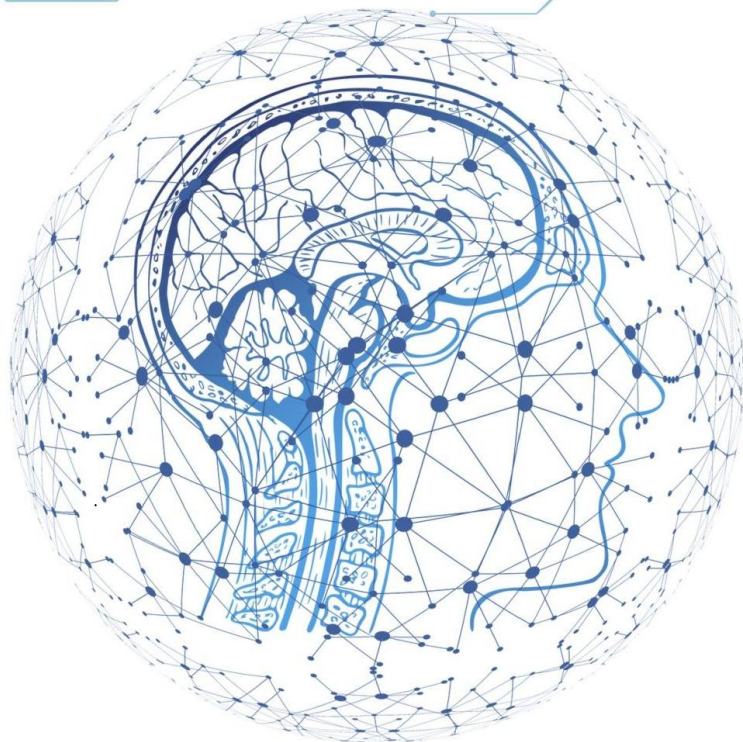
- Tốc độ cao cho deep learning
- Hiệu quả năng lượng tốt
- Phù hợp với mô hình lớn (CNN, Transformer)
- Tích hợp tốt với hệ sinh thái Google

■ Nhược điểm:

- Không linh hoạt như CPU/GPU
- Chỉ tối ưu cho một số loại phép toán
- Khó debug hơn
- Phụ thuộc vào framework (chủ yếu TensorFlow)

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Quy trình làm việc với Keras

Workflow chuẩn với Keras

1. Chuẩn bị dữ liệu
2. Định nghĩa mô hình
3. Compile mô hình
4. Train với `fit()`
5. Đánh giá với `evaluate()`
6. Dự đoán với `predict()`

■ Tư duy quan trọng

- Keras cung cấp một pipeline rất chuẩn
- Khi hiểu pipeline này, các mô hình khác nhau chỉ là biến thể của cùng một quy trình

Bước 1 – Chuẩn bị dữ liệu

■ Chuẩn bị dữ liệu gồm

- Nạp dữ liệu
- Chia train / validation / test
- Chuẩn hóa hoặc tiền xử lý
- Biến đổi shape nếu cần
- Tạo batch

■ Ví dụ

- Ảnh: chuẩn hóa pixel về $[0,1]$
- Văn bản: tokenize / vectorize
- Chuỗi thời gian: tạo windows / timesteps

Bước 2 – Định nghĩa mô hình

- **Có nhiều cách định nghĩa model**
 - Sequential API
 - Functional API
 - Model subclassing
- **Ví dụ tư duy**
 - Input layer
 - Hidden layers
 - Output layer
- **Với bài toán đơn giản**
 - Sequential thường là cách dễ nhất

Bước 3 – Compile mô hình

■ **Compile là bước cấu hình cách học**

- Chọn optimizer: cách cập nhật trọng số
- Chọn loss: thước đo sai số để tối ưu
- Chọn metrics: chỉ số để quan sát quá trình học

■ **Ví dụ**

- `optimizer="rmsprop"`
- `loss="sparse_categorical_crossentropy"`
- `metrics=["accuracy"]`

Bước 4 – Train với fit()

■ fit() thực hiện

- Chạy nhiều epoch
- Chia dữ liệu thành mini-batches
- Forward pass
- Tính loss
- Backpropagation
- Cập nhật weights

■ Thông số thường gặp

- **epochs**: Là số lần toàn bộ tập dữ liệu huấn luyện được đưa qua mô hình.
- **batch_size**: Là số lượng mẫu dữ liệu được đưa vào mô hình trong mỗi lần cập nhật trọng số.
- **validation_data**: Là tập dữ liệu dùng để đánh giá mô hình trong quá trình huấn luyện (không dùng để cập nhật trọng số). Thường có dạng (X_val, y_val).

■ Kết quả

- Trả về history của loss và metric theo từng epoch

Bước 5 và 6 – evaluate() và predict()

■ **evaluate()**

- Đánh giá mô hình trên tập kiểm tra
- Trả về loss và metrics

■ **predict()**

- Sinh dự đoán cho dữ liệu mới
- Cho xác suất hoặc output của model

■ **Phân biệt**

- fit() để học (để huấn luyện model)
- evaluate() để chấm điểm (để đánh giá model)
- predict() để suy luận (để dự đoán)

Sequential API – cách bắt đầu dễ nhất

■ Sequential phù hợp khi

- Mô hình là chuỗi layer tuyến tính
- Mỗi layer có một đầu vào, một đầu ra: Mỗi layer chỉ nhận đúng 1 tensor đầu vào và tạo ra 1 tensor đầu ra.
- Không có nhánh rẽ, không có nhiều input/output

■ Ưu điểm

- Viết nhanh, dễ hiểu
- Rất phù hợp cho học tập và prototype (thử nghiệm)

■ Hạn chế

- Không linh hoạt bằng Functional API

- **Sequential** phù hợp khi mô hình đơn giản, đi thẳng từ đầu vào đến đầu ra theo từng lớp.
- Nếu mô hình bắt đầu có nhiều nhánh, nhiều input/output hoặc kết nối phức tạp, hãy dùng **Functional API** của Keras thay vì **Sequential**.

Functional API – khi cần mô hình linh hoạt hơn

■ **Functional API phù hợp khi**

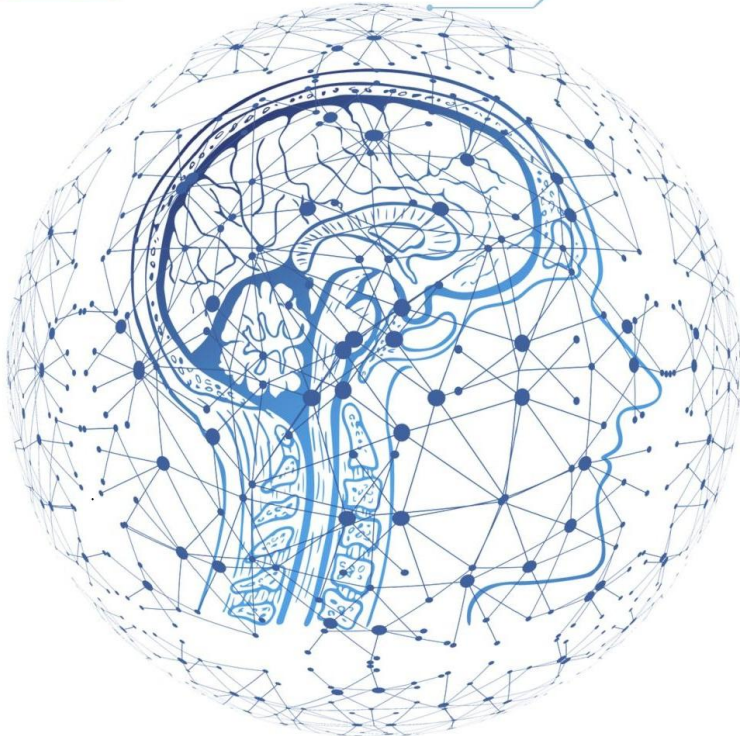
- Có nhiều input
- Có nhiều output
- Có kiến trúc phi tuyến
- Có nhánh rẽ hoặc chia sẻ layer

■ **Ý tưởng**

- Model được mô tả như một đồ thị các layer
- Linh hoạt hơn nhưng vẫn có cấu trúc rõ ràng

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Tạo layer và model mới bằng subclassing

Subclassing là gì?

■ Subclassing (Tạo lớp con)

- Tự định nghĩa class kế thừa từ **Layer** hoặc **Model**
- Tự viết logic tính toán trong **call()**

■ Khi nào dùng?

- Khi layer có phần xử lý riêng
- Khi mô hình không thuận tiện biểu diễn bằng Sequential / Functional API
- Khi cần kiểm soát hành vi chi tiết hơn

■ Lợi ích

- Linh hoạt mạnh
- Có thể đóng gói phần xử lý tái sử dụng

Một custom layer thường có gì?

■ Thành phần chính

- `__init__()` để khai báo tham số
- `build()` để tạo weights khi biết kích thước của dữ liệu đầu vào (shape input)
- `call()` để định nghĩa cách tính toán lan truyền xuôi (forward computation)

■ Ý tưởng quan trọng

- Layer không chỉ là hàm
- Layer còn mang trạng thái, đặc biệt là **trainable weights**

■ Ví dụ tư duy

- Dense layer tự viết
- Custom normalization: tự xây dựng cách chuẩn hóa dữ liệu
- Feature transformation riêng: tự định nghĩa cách biến đổi đặc trưng đầu vào trước khi đưa vào model.
 - Ví dụ:
 - Biến đổi phi tuyến
 - Kết hợp nhiều feature theo cách riêng
 - Áp dụng embedding, encoding đặc biệt

Layer subclassing và Model subclassing

■ Layer subclassing

- Dùng để tạo một khối xử lý dữ liệu

■ Model subclassing

- Dùng để ghép nhiều layer thành mô hình hoàn chỉnh

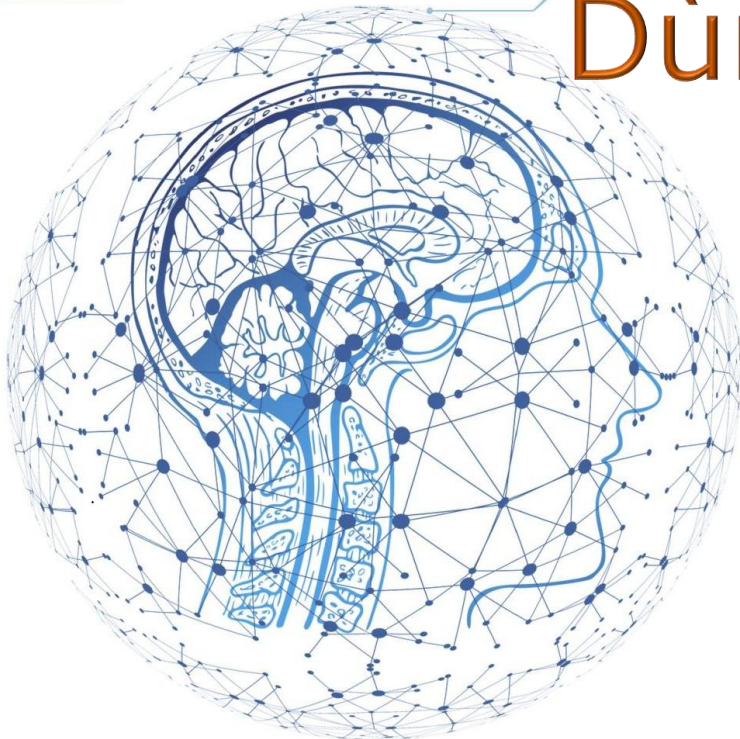
■ Model là một Layer đặc biệt

- Có thể chứa nhiều layer con
- Có thể **train, evaluate, save**

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Dùng các vòng lặp huấn luyện
và đánh giá có sẵn



Các vòng lặp có sẵn – Built-in loops

■ Built-in loops là

- Cơ chế huấn luyện và đánh giá có sẵn của Keras
- Dựa trên các lệnh quen thuộc như `fit()`, `evaluate()`, `predict()`

■ Ưu điểm

- Nhanh
- Ngắn gọn
- Ổn định
- Ít lỗi

■ Phù hợp khi

- Bài toán có workflow chuẩn
- Loss và metric không quá đặc biệt
- Không cần can thiệp sâu vào từng step

Dữ liệu đầu vào cho built-in loops

- **Keras hỗ trợ nhiều kiểu dữ liệu**
 - NumPy arrays
 - TensorFlow tensors
 - `tf.data.Dataset`
 - Python generators
- **Ý nghĩa**
 - Workflow dữ liệu rất linh hoạt
 - Có thể đi từ học tập đơn giản đến pipeline production
- **`tf.data.Dataset` đặc biệt hữu ích khi**
 - Dữ liệu lớn
 - Cần pipeline đọc/biến đổi hiệu quả
 - Cần shuffle, batch, prefetch

Callback – công cụ điều khiển quá trình train

■ Callback là gì?

- Cơ chế chen logic vào quá trình huấn luyện

■ Ví dụ callback phổ biến

- EarlyStopping
- ModelCheckpoint
- LearningRateScheduler
- ReduceLROnPlateau

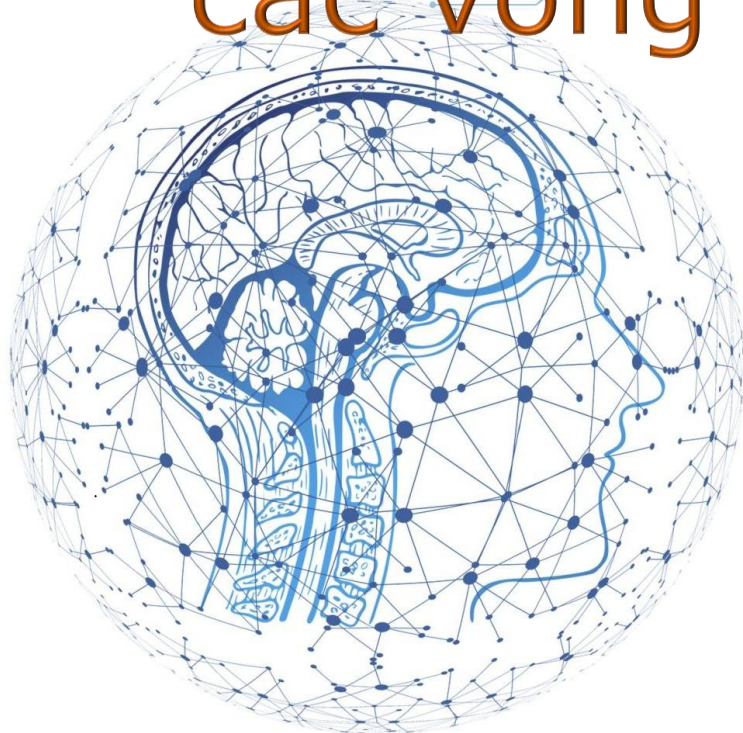
■ Tác dụng

- Dừng train khi không cải thiện
- Lưu model tốt nhất
- Điều chỉnh learning rate
- Tăng tính tự động cho training

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Tự xây dựng các vòng lặp huấn luyện và đánh giá



Khi nào cần tự xây dựng training loop?

■ Cần tự viết training loop khi

- Loss đặc biệt
- Logic cập nhật trọng số đặc biệt
- Nhiều bước tối ưu trong một batch
- Mô hình có hành vi học không chuẩn

■ Thành phần thường dùng

- `tf.GradientTape()`
- Forward pass
- Tính loss
- Tính gradient
- `optimizer.apply_gradients()`

Vai trò của `tf.GradientTape()`

- **GradientTape dùng để**
 - Ghi lại các phép toán trong forward pass
 - Tính gradient của loss theo weights
- **Quy trình**
 - Mở tape
 - Tính output và loss
 - Gọi tape để lấy gradients
 - Dùng optimizer cập nhật weights
- **Ý nghĩa**
 - Giúp chúng ta thấy rõ `fit()` đang làm gì phía sau

Built-in loop vs. custom loop

■ Built-in loop

- Nhanh triển khai
- Code ngắn
- Dễ bảo trì
- Phù hợp đa số bài toán

■ Custom loop

- Linh hoạt tối đa
- Kiểm soát sâu
- Tốn công hơn
- Dễ phát sinh lỗi hơn

Nguyên tắc chọn

- Bắt đầu bằng built-in
- Chỉ custom khi thực sự cần

Các lỗi người mới học thường gặp

1. Nhầm giữa loss và metric
 2. Nhầm evaluate() với predict()
 3. Không chú ý shape của dữ liệu
 4. Dùng sai loss cho loại nhãn
 5. Custom quá sớm khi built-in đã đủ
- **Thông điệp**
 - Hiểu workflow quan trọng hơn học thuộc API

Tổng kết chương

■ Những ý cần nhớ

- TensorFlow là nền tảng tính toán cho deep learning
- Keras là API mức cao, thân thiện và mạnh
- Workflow chuẩn: prepare → build → compile → fit → evaluate → predict
- Có 3 cách xây model phổ biến: Sequential, Functional, subclassing
- Built-in loop nên là lựa chọn mặc định
- Custom loop dùng khi cần kiểm soát sâu hơn

Ghi nhớ: TensorFlow là engine, Keras là giao diện xây dựng, còn workflow là xương sống của mọi bài toán deep learning

Hoạt động trên lớp

■ Hoạt động 1: hỏi nhanh

- `fit()` dùng để làm gì?
- `evaluate()` khác `predict()` ở điểm nào?

■ Hoạt động 2: thảo luận nhóm 3 phút

- Một mô hình nhiều đầu vào thì Sequential có còn phù hợp không?

■ Hoạt động 3: câu hỏi phản biện

- “Keras quá đơn giản nên không phù hợp cho bài toán thực tế phức tạp.”
Bạn đồng ý hay không? Vì sao?

Câu hỏi thảo luận

- **Câu 1**

Vì sao Keras phù hợp với người mới học nhưng vẫn hữu ích cho chuyên gia?

- **Câu 2**

Khi nào nên dùng Sequential, khi nào nên dùng Functional API?

- **Câu 3**

Vì sao built-in loop thường được ưu tiên hơn custom loop?

- **Câu 4**

Nếu một bài toán có loss rất đặc biệt, vì sao fit() có thể không còn đủ?

